A Case Study of Staged Metric-Gated GRPO for Visual Numeric Reasoning

Kunal Ghosh
Purdue University
West Lafayette, Indiana, USA
ghosh178@purdue.edu
kg.aero@gmail.com

Abstract

We present a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning. The pipeline combines four components: a phase-wise curriculum over filtered MathVista subsets, a strict <REASONING> / <SOLUTION> response contract, a checkpoint-metric controller that preserves structure-oriented reward floors under formatting or completion degradation and increases correctness weight after structural metrics stabilize and exact-match progress stalls, and alias-based checkpoint selection to address non-monotonic checkpoint quality. The resulting trajectory follows a consistent two-stage progression: smoke-phase runs on `testmini` stabilize parseability and output structure, and larger-split Phase C fine-tuning on `test`, evaluated on `testmini`, delivers the main correctness improvement. Under the checkpoint-side evaluation protocol used for continuation, the best large Phase C checkpoint and the best dedicated Phase D checkpoint each achieve exact-match and tolerance accuracy of 0.75, compared with 0.375 for a pre-refactor baseline, while maintaining parseability 1.0 and zero truncation on the monitored subset. These results indicate that curriculum filtering, metric-gated reward adaptation, and checkpoint-aware continuation support a stable, structure-first RL fine-tuning pipeline for multimodal numeric question answering under constrained compute.

Code: https://github.com/kgaero/RL_GSPO_Qwen2.5VLM

1 Introduction

Reinforcement-learning fine-tuning for language and vision-language models is often summarized as a direct mapping from reward design to improved accuracy. In multimodal settings, however, the optimization problem is more complex (Yuan et al., 2025). Before correctness reward becomes informative, the policy must already emit outputs that are parseable, non-truncated, and structurally compatible with the evaluator (Shen et al., 2025). If those preconditions are unstable, RL updates can be difficult to interpret because an apparent correctness failure may in fact be a formatting or completion failure (Shen et al., 2025).

This paper investigates a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning using `Qwen2.5-VL-7B-Instruct` as the base VLM. The pipeline leaves the GRPO objective unchanged and augments fine-tuning with filtered curriculum stages, a strict response contract, rule-based reward adaptation keyed to checkpoint metrics, and checkpoint aliases that allow continuation from the best structural or composite checkpoint rather than always from the most recent one. We analyze how this fine-tuning pipeline produces a structure-first and correctness-later trajectory for multimodal numeric question answering.

The implementation is designed to be auditable: controller states are saved, checkpoint aliases are explicit, reward weights are persisted, and the main milestones are recoverable from checkpoint-side evaluation logs. This instrumentation is valuable in constrained-compute RL settings, where structural failures dominate early training and continuation decisions materially affect outcomes.

Figure 1 summarizes the loop. A base VLM is prepared with LoRA adapters, fine-tuned on a stage-filtered phase mixture, evaluated checkpoint by checkpoint, and then fed back into a controller

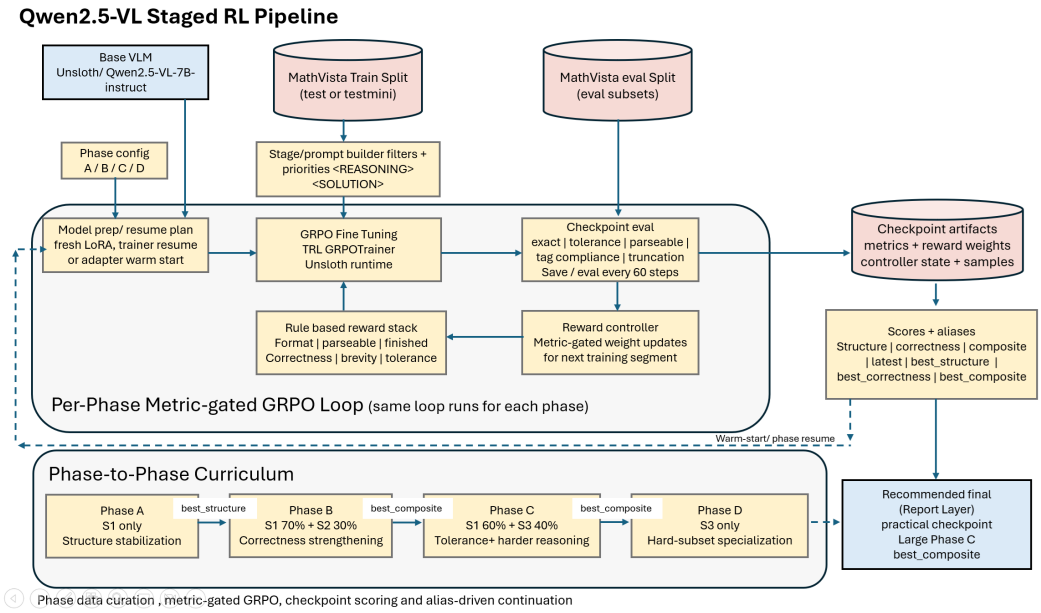

Figure 1: Overview of the staged metric-gated GRPO loop. The pipeline couples phase-specific data curation, GRPO fine-tuning, checkpoint-side evaluation, reward-weight adaptation, and alias-based continuation. The method studied here is the fine-tuning pipeline around GRPO rather than a new GRPO loss.

that updates reward weights after each checkpoint evaluation. Table 1 shows that the curriculum is implemented as overlapping filtered views over the same MathVista split rather than as disjoint datasets.

Contributions.

1. We document a staged GRPO fine-tuning pipeline for visual numeric reasoning that explicitly prioritizes structure stabilization before the main correctness gains.
2. We formalize the metric-gated reward controller and checkpoint aliasing logic so they can be read as an auditable algorithm rather than as scattered implementation details.
3. We characterize the checkpoint-side training trajectory used for continuation decisions, including the 0.75 exact-match milestone reached by the best large Phase C and dedicated Phase D checkpoints.

2 Related Work

RL fine-tuning and preference optimization. Recent RLHF and preference-optimization work has primarily focused on new objectives or more efficient learning regimes. DeepSeekMath introduced GRPO as a group-relative alternative to PPO for mathematical reasoning (Shao et al., 2024). SPPO and Asynchronous RLHF studied, respectively, self-play preference optimization and faster off-policy online RLHF (Wu et al., 2025; Noukhovitch et al., 2025). More recent multimodal post-training studies have adapted GRPO- or RL-style optimization to vision-language reasoning through cold-start supervision, staged reinforcement learning, and explicit reasoning incentives (Huang et al., 2025; Wei et al., 2025; Chen et al., 2025). Our contribution is complementary: we leave the underlying GRPO objective unchanged and study how a staged fine-tuning pipeline combines GRPO with curriculum scheduling, metric-gated reward adaptation, and checkpoint-selection logic in a failure-prone multimodal setting.

Multimodal mathematical reasoning. MathVista highlights the difficulty of mathematical reasoning in visual contexts and serves as a benchmark for diagram, chart, table, and numeric question

answering behavior (Lu et al., 2024). Recent work has addressed this problem with larger synthetic corpora, progressive curricula, staged reinforcement learning, or more explicit visual reasoning mechanisms (Zhang et al., 2025; Qi et al., 2025; Liu et al., 2025; Yuan et al., 2025). This paper focuses on the training-system side of the problem: a standard pretrained VLM paired with staged curricula, metric-gated reward control, and checkpoint-aware continuation that prioritize structural reliability before stronger correctness pressure.

Curriculum and adaptive reward control. Our approach is closest in spirit to curriculum and adaptive-control approaches that treat fine-tuning as a sequence of regimes rather than as one stationary objective. Recent multimodal reasoning work has explored phased reinforcement learning, progressive curriculum RL, and rubric-based curriculum design as mechanisms for stabilizing difficult reasoning behavior (Liu et al., 2025; Yuan et al., 2025; Chen et al., 2026). The distinctive feature here is the granularity of adaptation: checkpoint metrics trigger reward changes directly within the fine-tuning pipeline.

3 Problem Setup and Design Goals

We study a policy $\pi_\theta(c \mid x)$ that maps a visual question-answering input x to a completion c. Our method does not treat success as a single scalar notion of accuracy. Instead, it decomposes success into a sequence of preconditions:

1. the model must emit the required tagged structure,
2. the final answer must be parseable,
3. the answer must finish within the completion budget, and
4. the parsed answer must be correct under exact or tolerance-based matching.

The central question is therefore not only how to optimize correctness, but also when correctness reward becomes informative enough to emphasize. If structure is unstable, correctness reward is sparse because malformed or truncated outputs never reach a meaningful matching stage.

The implementation makes three concrete design choices.

- **Easy-to-hard data staging.** The policy first sees easier integer-valued numeric problems before harder float and geometry-heavy subsets are emphasized.
- **Metric-gated rewards.** Reward weights change only after checkpoint evaluation, using measured failure modes rather than a fixed epoch schedule.
- **Alias-based continuation.** Checkpoint continuation is based on structure, correctness, or composite aliases rather than assuming the latest checkpoint is best.

Taken together, these choices define the central design logic of the method.

4 Method

4.1 Task, response contract, and phase structure

The code targets MathVista-style visual question answering in which the answer is primarily numeric and free-form. Each prompt is wrapped with a strict output contract:

```
<REASONING> ...  </REASONING>
<SOLUTION> single numeric answer </SOLUTION>
```

This contract matters because both the reward functions and the evaluator depend on extracting a unique solution string from the completion. The pipeline also contains a separate multi-choice scaffold, but that branch is disabled by default and is not part of the main evidence.

The curriculum is organized into stages and phases. Stages are filtered subsets over the same dataset. In the default configuration:

Filter	S1 Easy Numeric	S2 Medium / Float	S3 Hard Numeric
Stage ID	stage1_easy_numeric	stage2_float_numeric	stage3_hard_numeric
Goal	easy numeric stabilization	moderate precision and broader numeric coverage	harder multistep numeric reasoning
Question type	free_form	free_form	free_form
Answer type	integer	integer, float	integer, float
Language	english	english	english
Answer mode	numeric_free_form	numeric_free_form	numeric_free_form
Context families	synthetic scene table natural image	table chart plot scientific figure natural image	geometry diagram plot scientific figure abstract scene
Skills (any)	arithmetic reasoning statistical reasoning numeric commonsense	arithmetic reasoning statistical reasoning scientific reasoning numeric commonsense	geometry reasoning algebraic reasoning scientific reasoning arithmetic reasoning statistical reasoning
Hard only	False	False	True
Priority bias	ctx: synthetic scene ctx: table skills: arithmetic reasoning skills: statistical reasoning skills: numeric commonsense grades: elementary school grades: daily life	ctx: table ctx: chart ctx: plot ctx: scientific figure skills: statistical reasoning skills: scientific reasoning skills: arithmetic reasoning	ctx: geometry diagram ctx: scientific figure ctx: plot skills: geometry reasoning skills: algebraic reasoning skills: scientific reasoning grades: high school grades: college

Table 1: Code-derived filter definitions for the three numeric curriculum stages used in the main training path. The stages overlap and are implemented as filtered views over the same MathVista split, not as fixed disjoint datasets.

- **Stage 1** restricts to easier English free-form integer questions on synthetic scenes, tables, and natural images.
- **Stage 2** broadens to integer and float answers, adding tables, charts, plots, and scientific figures.
- **Stage 3** emphasizes harder geometry, scientific-figure, plot, and abstract-scene problems.

Phases then mix these stages with different reward emphases:

- **Phase A**: Stage 1 only, structure stabilization.
- **Phase B**: 0.7 Stage 1 + 0.3 Stage 2, early correctness strengthening.
- **Phase C**: 0.6 Stage 2 + 0.4 Stage 3, harder reasoning and tolerance reward.
- **Phase D**: Stage 3 only, hard-subset specialization with a longer completion budget.

4.2 Reward decomposition

For a completion c, the method computes six per-sample reward components:

$$R(c) = \sum_{j=1}^{6} w_j r_j(c), \tag{1}$$

where the components correspond to `format`, `parseable`, `finished`, `correctness`, `brevity`, and `tolerance`. The initial weights are phase-specific. Phase A places relatively high weight on structure and completion; Phase C raises correctness and activates tolerance reward.

The implementation never allows structure rewards to decay fully to zero. The guiding design principle is that, even when exact match becomes the main target, structural validity remains a necessary condition for receiving informative reward. The exact code-aligned reward definitions are reproduced in the supplement.

4.3 METRIC-GATED REWARD CONTROL

The reward controller updates weights after each checkpoint evaluation using explicit rules derived from observed metrics. Let p denote parseable-answer rate, s solution-tag compliance, r reasoning-tag compliance, m malformed rate, t truncation rate, and e exact match. The controller applies the following guards:

$$p < 0.85 \Rightarrow w_{\text{parseable}} \leftarrow \max(w_{\text{parseable}}, 0.75), \tag{2}$$

$$(s < 0.90) \ \vee \ (r < 0.88) \ \vee \ (m > 0.10) \Rightarrow w_{\text{format}} \leftarrow \max(w_{\text{format}}, 0.75), \tag{3}$$

$$(t > 0.15) \ \vee \ (\bar{\ell} > 0.85L) \Rightarrow w_{\text{finished}} \leftarrow w_{\text{finished}} + 0.25, \tag{4}$$

where $\bar{\ell}$ is average completion length and L is the completion budget.

Correctness is escalated only when structure is already stable. Stability requires

$$p \geq 0.92, \quad s \geq 0.95, \quad r \geq 0.93, \quad m \leq 0.05, \quad t \leq 0.08, \tag{5}$$

for a two-checkpoint window. If that condition holds and exact-match improvement over the window is below 0.02, the controller increases correctness weight by 0.50. All weights are then clamped to configured bounds. The rule set is intentionally simple enough to audit directly in saved checkpoint artifacts.

4.4 CHECKPOINT-SIDE EVALUATION AND ALIAS-BASED SELECTION

Instead of treating the most recent checkpoint as the default continuation point, the pipeline evaluates every saved checkpoint and computes three aggregate scores:

$$S_{\text{struct}} = 0.35\,p + 0.20\,s + 0.20\,r - 0.15\,m - 0.10\,t, \tag{6}$$

$$S_{\text{corr}} = 0.70\,e + 0.20\,a_{\text{tol}} + 0.10\,p, \tag{7}$$

$$S_{\text{comp}} = 0.45\,e + 0.15\,a_{\text{tol}} + 0.15\,p + 0.10\,s + 0.05\,r - 0.05\,m - 0.05\,t, \tag{8}$$

where a_{tol} is tolerance accuracy. These scores back four aliases: `latest`, `best_structure`, `best_correctness`, and `best_composite`. Cross-phase continuation typically resumes from `best_structure` or `best_composite`, while same-phase restarts may use full trainer resume from `latest`. This distinction is important in the implementation because the code supports both full trainer-state restoration and LoRA-only warm start.

5 EXPERIMENTAL SETTING AND EVALUATION PROTOCOL

Dataset and model. We use the Hugging Face release of MathVista[1] and load `Qwen2.5-VL-7B-Instruct`[2] as the base VLM. For context we cite the MathVista benchmark paper (Lu et al., 2024) and the Qwen2.5-VL technical report (Bai et al., 2025).

Runtime profile. The main reported runs use the conservative `kaggle_t4` profile: 4-bit loading, LoRA rank 8, max sequence length 1280, image size 336, gradient accumulation 4, two sampled generations per prompt during RL fine-tuning, max prompt length 320, and max completion length 64 (or 96 in Phase D). The trainer uses TRL's `GRPOTrainer` with sequence-level importance sampling and `dr_grpo` loss.

Checkpoint-side evaluation protocol. All quantitative values reported in this paper come from checkpoint-side metrics generated by the code during training and continuation. In the smoke phases, the pipeline trains on `testmini` and also evaluates on `testmini` in order to diagnose and correct structural failures efficiently. In the larger continuation, the pipeline trains on `test` and evaluates on `testmini`; the headline 0.75 results come from this larger continuation regime and the dedicated Phase D continuation that follows it. Under the Kaggle profile, evaluation uses one sampled completion per prompt and caps each subset at two examples, matching the runtime-constrained operating regime used for the reported runs.

[1] https://huggingface.co/datasets/AI4Math/MathVista
[2] https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct

Parameter	Default	New Value	Function Name	Phase
max_seq_length	16384	1280	FastVisionModel.from_pretrained	all
image_size	512	336	build_stage_dataset	all
gpu_memory_utilization	0.8	0.65	FastVisionModel.from_pretrained	all
LoRA config (lora_rank / max_lora_rank / lora_alpha)	16 / None / 16	8 / 8 / 8	FastVisionModel.get_peft_model	all
compilation_config	unset	level 3 PIECEWISE	FastVisionModel.from_pretrained	all
Training budget (gradient_accumulation_steps / num_generations)	2 / 4	4 / 2	GRPOConfig	all
Prompt budget (max_prompt_length / max_completion_length)	1024 / 256	320 / 64	GRPOConfig	A–C, E
Eval sampling (num_samples_per_prompt / max_eval_examples_per_subset)	4 / None	1 / 2	SamplingParams	all
Phase D completion budget (max_completion_length)	320	96	GRPOConfig / SamplingParams	D only

Table 2: Code-derived implementation summary of the low-memory profile used by the successful runs. The **Function Name** column lists the concrete runtime method or config class that consumes the setting, rather than the config-builder that defines its default. The smoke experiments and the larger continuation therefore share the same Kaggle T4-safe operating regime.

Table 3: Milestone checkpoints under the checkpoint-side evaluation protocol used by our runs. These are the recorded metrics used by the continuation logic.

Checkpoint	EM	Tol	Parse	Malf.	Trunc.	Comp.
Pre-refactor baseline	0.375	0.375	0.641	0.359	0.242	0.405
Smoke Phase A best	0.500	0.500	0.500	0.500	0.500	0.400
Smoke Phase B best	0.500	0.500	1.000	0.000	0.000	0.600
Smoke Phase C best	0.500	0.500	1.000	0.000	0.000	0.600
Large Phase C best	**0.750**	**0.750**	**1.000**	**0.000**	**0.000**	**0.750**
Dedicated Phase D best	**0.750**	**0.750**	**1.000**	**0.000**	**0.000**	**0.750**

Reproducibility. The implementation saves checkpoint-side artifacts including metrics, subset metrics, reward weights, controller state, controller decision logs, prompt-level records, and sample-level JSONL traces. It also includes 28 unit tests covering configuration, data filtering, parsing, checkpointing, evaluation aggregation, controller behavior, and runtime compatibility.

6 Results

6.1 Structure improves before correctness in the training trajectory

Table 3 shows the progression most clearly in the checkpoint-side metrics. The pre-refactor baseline reaches only 0.641 parseability with 0.359 malformed rate and 0.242 truncation. Smoke Phase A shortens outputs, but structure remains unstable. A substantial structural transition appears in Smoke Phase B: parseability, reasoning-tag compliance, and solution-tag compliance all reach 1.0, while malformed outputs and truncation both fall to zero. Exact match, however, remains at 0.5. These results indicate that the smoke phases on `testmini` primarily stabilize structure, whereas the larger-split Phase C run on `test` delivers the main correctness improvement.

Figure 2 visualizes this transition. Once structural metrics saturate, subsequent progress shifts from improving output validity to improving the correctness of already valid outputs. This interpretation is consistent with the controller design: early checkpoints allocate reward mass to formatting,

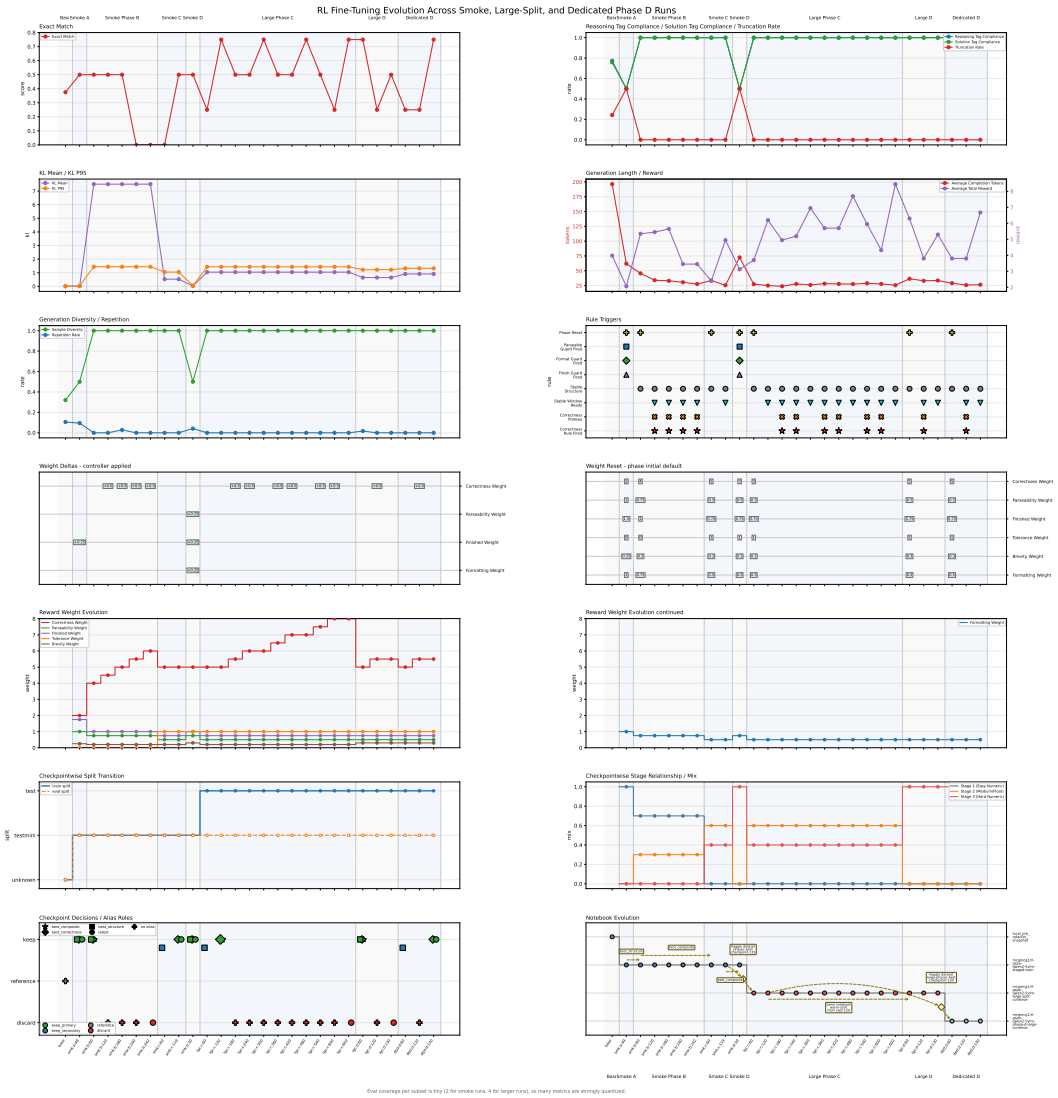

Figure 2: Training evolution across milestone checkpoints. The main qualitative pattern is structure first, correctness later: structural metrics stabilize by the smoke Phase B / Phase C regime, while the 0.75 exact-match milestone appears only after larger continuation.

Table 4: Comparison between the pre-refactor baseline snapshot and the best large Phase C checkpoint.

Core Accuracy			
Eval	**Base**	**Trained**	**% Improvement**
Exact Match	**37.5%**	**75.0%**	**+100.0%**
Tolerance Accuracy	37.5%	75.0%	**+100.0%**
Structure / Completion			
Eval	**Base**	**Trained**	**% Improvement**
Parseable Rate	64.1%	100.0%	**+56.1%**
Reasoning Tag Compliance	**77.3%**	**100.0%**	**+29.3%**
Solution Tag Compliance	**75.8%**	**100.0%**	**+32.0%**
Well-Formed Rate	64.1%	100.0%	**+56.1%**
Malformed Rate (lower better)	35.9%	0.0%	**+100.0%**
Completion Success Rate	75.8%	100.0%	**+32.0%**
Truncation Rate (lower better)	24.2%	0.0%	**+100.0%**
Generation			
Eval	**Base**	**Trained**	**% Improvement**
Average Completion Tokens (lower better)	196.3	25.0	**+87.3%**
Repetition Rate (lower better)	10.6%	0.0%	**+100.0%**
Sample Diversity	32.0%	100.0%	**+212.2%**
Scores / Reward			
Eval	**Base**	**Trained**	**% Improvement**
Average Total Reward	3.98	6.2	**+55.7%**
Structure Score	45.2%	75.0%	**+65.8%**
Correctness Score	40.2%	77.5%	**+93.0%**
Composite Score	40.5%	75.0%	**+85.0%**

Source: Pre-Refactor Baseline (`base`) vs Large Phase C checkpoint-120 (`lgc:c:120`) from the checkpoint tables generated by the fine-tuning pipeline.

parseability, and clean completion, whereas later checkpoints inherit those gains and receive greater correctness pressure.

6.2 THE 0.75 MILESTONE APPEARS IN LARGER CONTINUATION

The strongest checkpoints in our runs are the large-split Phase C model at checkpoint 120 and the dedicated Phase D model at checkpoint 130. Relative to the pre-refactor baseline snapshot, the large Phase C checkpoint moves exact match from 0.375 to 0.75, tolerance accuracy from 0.375 to 0.75, parseability from 0.641 to 1.0, and average completion length from 196.3 to 25.0 tokens. Table 4 summarizes that before-versus-after comparison.

The dedicated Phase D run matches, rather than exceeds, the best Phase C score. This indicates that hard-subset specialization is a viable continuation path, while the mixed Stage 2/3 continuation remains the strongest route to the 0.75 milestone within the reported runs.

6.3 CHECKPOINT-SIDE RESULTS MOTIVATE ALIAS-BASED SELECTION

The checkpoint-side results also clarify the role of alias-based checkpoint selection. In Smoke Phase B, the best composite checkpoint reaches exact match 0.5 with perfect structure, while the latest checkpoint regresses to exact match 0.0 despite retaining parseability and tag compliance at 1.0. In the same-notebook Phase D branch, the best checkpoint reaches 0.75 exact match but the latest checkpoint ends at 0.5. This non-monotonicity in the checkpoint lineage explains why explicit aliases are preferable to always resuming from `latest`.

6.4 THE CONTROLLER AUDIT IS CONSISTENT WITH THE INTENDED POLICY

The saved controller audit covers 26 evaluated checkpoints across seven phase resets. Its main descriptive result is that the rules fire in the intended order. Parseability, formatting, and finish guards fire only twice each, concentrated in the structurally weak part of the trajectory. Correctness escalation fires 12 times, but only after structure is already stable. Notably, six of those correctness escalations occur on checkpoints where exact match has regressed rather than merely plateaued. This behavior makes the controller's operating bias explicit: once structure is considered stable, the system responds to weak accuracy progress by increasing correctness pressure rather than by re-raising structure rewards.

Figure 2 places that audit trail back into the checkpoint lineage.

6.5 THE 0.75 MILESTONE IS OBTAINED UNDER THE SAME LOW-MEMORY PROFILE

Table 2 and the runtime-tuning tables show that the large continuation does not coincide with a more permissive hardware regime. The successful smoke and large-split runs share the same Kaggle T4 profile: sequence length 1280 instead of 16384, image size 336 instead of 512, LoRA rank 8 instead of 16, and two sampled generations instead of four. The trade-off is a shorter reasoning budget and lower generation diversity. The benefit is a stable single-GPU operating regime that supports both early curriculum validation and later continuation without changing the underlying VRAM regime.

7 LIMITATIONS AND REPRODUCIBILITY

- This study was developed and run in a resource-constrained Kaggle T4 environment, and larger-scale experiments are required to determine how far the same curriculum-based RL fine-tuning pipeline stabilizes under longer runs, larger evaluation budgets, and broader hardware regimes.
- The monitored evaluation subset is tiny in the Kaggle profile, so the quantitative results should be interpreted within that runtime-constrained operating regime.
- The split protocol is phase-dependent: smoke phases use `testmini` to rapidly diagnose and stabilize structure, while the main correctness result comes from larger-split Phase C fine-tuning on `test` with evaluation on `testmini`.
- The codebase studies one base model family and one principal hardware profile, and it does not yet include the seed sweeps or controlled ablations needed to validate each component of the fine-tuning pipeline in isolation.
- The multi-choice branch (Phase E) and multilingual robustness stage are scaffolded but are not part of the main evidence.
- Even with these limits, the observed structure-first and correctness-later trajectory suggests that curriculum-based staged RL fine-tuning pipelines are a promising research direction for multimodal numeric reasoning and merit further scale-up.

The study remains comparatively reproducible. It logs controller decisions at every evaluated checkpoint, persists multiple checkpoint-selection aliases, saves sample-level traces, and includes unit tests for the most failure-prone logic. The supplement expands this record with explicit reward equations, evaluation formulas, controller thresholds, and transfer semantics.

8 CONCLUSION

We presented a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning in a resource-constrained setting. The checkpoint-side results indicate a consistent progression: structural validity must become reliable before correctness reward becomes consistently informative. A phase-wise curriculum, metric-gated reward controller, and alias-based checkpoint selection transform a malformed, long-output baseline into checkpoints that reach exact match 0.75 while preserving parseability 1.0 and zero truncation on the monitored subset. Smoke-phase runs stabilize structure on `testmini`, and larger-split Phase C fine-tuning on `test` delivers the principal correctness improvement. These findings support staged curriculum-based RL fine-tuning pipelines as a promising direction for multimodal numeric reasoning under practical compute constraints.

REFERENCES

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, Junyang Lin, et al. Qwen2.5-vl technical report, 2025. URL https://arxiv.org/abs/2502.13923.

Shuang Chen, Yue Guo, Zhaochen Su, Yafu Li, Yulun Wu, Jiacheng Chen, Jiayu Chen, Weijie Wang, Xiaoye Qu, and Yu Cheng. Advancing multimodal reasoning: From optimized cold start to staged reinforcement learning, 2025. URL https://arxiv.org/abs/2506.04207.

Yukun Chen, Jiaming Li, Longze Chen, Ze Gong, Jingpeng Li, Zhen Qin, Hengyu Chang, Ancheng Xu, Zhihao Yang, Hamid Alinejad-Rokny, Qiang Qu, Bo Zheng, and Min Yang. Rucl: Stratified rubric-based curriculum learning for multimodal large language model reasoning, 2026. URL https://arxiv.org/abs/2602.21628.

Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models, 2025. URL https://arxiv.org/abs/2503.06749.

Zeyu Liu, Yuhang Liu, Guanghao Zhu, Congkai Xie, Zhen Li, Jianbo Yuan, Xinyao Wang, Qing Li, Shing-Chi Cheung, Shengyu Zhang, Fei Wu, and Hongxia Yang. Infi-mmr: Curriculum-based unlocking multimodal reasoning via phased reinforcement learning in multimodal small language models, 2025. URL https://arxiv.org/abs/2505.23091.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.02255.

Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.18252.

Ji Qi, Ming Ding, Weihan Wang, Yushi Bai, Qingsong Lv, Wenyi Hong, Bin Xu, Lei Hou, Juanzi Li, Yuxiao Dong, and Jie Tang. Cogcom: A visual language model with chain-of-manipulations reasoning. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2402.04236.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Chuming Shen, Wei Wei, Xiaoye Qu, and Yu Cheng. Satori-r1: Incentivizing multimodal reasoning through explicit visual anchoring, 2025. URL https://arxiv.org/abs/2505.19094.

Lai Wei, Yuting Li, Kaipeng Zheng, Chen Wang, Yue Wang, Linghe Kong, Lichao Sun, and Weiran Huang. Advancing multimodal reasoning via reinforcement learning with cold start, 2025. URL https://arxiv.org/abs/2505.22334.

Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference optimization for language model alignment. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2405.00675.

Ruifeng Yuan, Chenghao Xiao, Sicong Leng, Jianyu Wang, Long Li, Weiwen Xu, Hou Pong Chan, Deli Zhao, Tingyang Xu, Zhongyu Wei, Hao Zhang, and Yu Rong. Vl-cogito: Progressive curriculum reinforcement learning for advanced multimodal reasoning, 2025. URL https://arxiv.org/abs/2507.22607.

Renrui Zhang, Xinyu Wei, Dongzhi Jiang, Ziyu Guo, Yichi Zhang, Chengzhuo Tong, Jiaming Liu, Aojun Zhou, Shanghang Zhang, Peng Gao, and Hongsheng Li. Mavis: Mathematical visual instruction tuning with an automatic data engine. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2407.08739.